

5. Raportowanie błędów, podejście TDD oraz testy jednostkowe

5.1. Raportowanie błędów w systemie śledzenia (Bug Tracking)

W ramach symulacji cyklu testowania oprogramowania, w systemie śledzenia błędów zarejestrowano i pomyślnie rozwiązano krytyczne zgłoszenia. Poniżej znajduje się wyciąg z rejestru błędów:

Przepełnienie bufora UART przy szybkich zmianach pozycji

- Opis: Podczas generowania dużej liczby punktów pomiarowych, aplikacja w Pythonie wysyłała ramki współrzędnych szybciej, niż Arduino było w stanie przetworzyć zapytanie `Serial.parseInt()`. Skutkowało to gubieniem znaków końca linii `\n` i błędnym pozycjonowaniem.
- Rozwiązanie: Wprowadzono pętlę blokującą w Pythonie, która bezwzględnie oczekuje na komunikat `"done"` z Arduino przed wysłaniem kolejnej ramki.

Wycinanie bezpośredniej fali (t_0) przy ujemnych wartościach `pre_ms`

- Opis: W funkcji `measure_ir_sweep`, przy próbie ustawienia ujemnego marginesu czasu przed impulsem, indeksowanie tablicy `t0 - pre` przyjmowało wartości spoza zakresu, co crashowało skrypt.
- Rozwiązanie: Dodano walidację typów i zabezpieczenie `max(0, t0 - pre)` gwarantujące nieprzekroczenie granic tablicy NumPy.

5.2. Metodyka Test-Driven Development (TDD) – Zapewnienie Jakości

W celu zapewnienia bezbłędnego działania algorytmów cyfrowego przetwarzania sygnałów (DSP), przed napisaniem właściwego kodu implementacji zastosowano technikę TDD (Czerwony-Zielony-Refaktor).

Wprowadzono założenie projektowe: funkcja odpowiedzialna za okienkowanie czasowe (bramkowanie odpowiedzi impulsowej) musi automatycznie dopasować długość okna wygładzającego Tukey'a do wyciętego fragmentu sygnału.

1. Faza Czerwona: Napisano test sprawdzający, czy długość okna jest identyczna z długością wyciętego sygnału. Test zakończył się niepowodzeniem (brak implementacji funkcji).
2. Faza Zielona: Zaimplementowano kod `window = sig.windows.tukey(len(hrir_left), alpha=0.25)` i przemnożenie sygnału. Test przeszedł pomyślnie.
3. Faza Refaktoru: Zoptymalizowano operację do postaci wektorowej NumPy (`hrir_left *= window`), zapewniając maksymalną wydajność CPU.

5.3. Testy Jednostkowe do Walidacji Oprogramowania

Stworzono dedykowany kod testów jednostkowych, który symuluje zachowanie algorytmów bez potrzeby podłączania fizycznego Arduino i głośników.

Wynik wykonania walidacji i testów jednostkowych oprogramowania:

```
...
-----
Ran 3 tests in 0.006s
OK
```

6. Optymalizacja

Za pomocą narzędzia cProfile zidentyfikowano główne punkty krytyczne (hotspots) w opracowanym programie. Największe obciążenie procesora generowała funkcja dekonwolucji ``scipy.signal.convolve`` realizowana w dziedzinie czasu dla sygnałów o częstotliwości 48 kHz. Optymalizację tego punktu można przeprowadzić poprzez zastąpienie splotu czasowego splotem w dziedzinie częstotliwości (``scipy.signal.fftconvolve``), co skróciłoby czas obliczeń ponad stukrotnie. Drugim zidentyfikowanym hotspotem były operacje wejścia/wyjścia (zapis plików WAV w pętli pomiarowej). Zoptymalizować to można poprzez buforowanie wyników w pamięci RAM i ich seryjny zapis na dysk po zakończeniu procedury ruchu.